School of Future Tech

Case Study Report

on

Railway Reservation System using C++

by

TEJAS TUSHAR CHAVAN

150096725185

Index

1. Introduction to the Case Study

Railway transportation is one of the most widely used modes of travel. Managing seat availability, bookings, and cancellations efficiently is crucial for smooth operations.

This case study focuses on designing and implementing a **Railway Reservation System using C++**. The system simulates real-world train booking operations through a menu-driven console application.

The project demonstrates how structured programming concepts such as arrays, loops, and conditional statements can be used to manage reservations logically and efficiently.

2. Problem Statement / Case Background (Abstract)

Background

Traditional reservation systems require proper validation to avoid overbooking or incorrect seat management. Even in simplified simulations, maintaining data consistency is important.

Abstract

This case study presents the design and implementation of a Railway Reservation System using C++. The system allows users to:

- View available trains
- Book seats
- Cancel reservations
- Generate billing based on seat pricing

Train data such as train number, train name, total seats, available seats, and price per seat are stored using arrays. The system ensures proper validation before booking or cancellation. The application runs in a loop until the user selects the exit option.

3. Case Study Design

The Railway Reservation System is designed as a simple structured program with the following components:

Data Storage

Train details are stored using arrays:

- Train Number
- Train Name
- Total Seats
- Available Seats
- Price per Seat

```
// Train Data
int trainNo[2] = {12951, 12002};
string trainName[2] = {"Mumbai Rajdhani", "Shatabdi Express"};
int totalSeats[2] = {25, 30};
int availableSeats[2] = {25, 30};
int pricePerSeat[2] = {1500, 1200};

int choice, number, seats;
```

Screenshot of array initialization section

= **Train Data Initialization using Arrays**

Control Flow Design

The system uses:

- `do-while` loop for repeated menu execution
- `switch-case` statement for user selection
- `if-else` conditions for validation

```
do {
    cout << "\n1. View Trains";
    cout << "\n2. Book Seat";
    cout << "\n3. Cancel Seat";
    cout << "\n4. Exit";
    cout << "\nEnter Choice: ";
    cin >> choice;
```

```
switch(choice) {

case 1: // View trains
    for(int i = 0; i < 2; i++) {
        cout << "\nTrain No: " << trainNo[i];
        cout << "\nTrain Name: " << trainName[i];
        cout << "\nAvailable Seats: " << availableSeats[i];
        cout << "\nPrice per Seat: Rs." << pricePerSeat[i];
        cout << "\n---------------------------\n";
    }
    break;
```

Screenshot of do-while loop and switch-case menu

= **Menu-Driven Control Structure**

4. Methods & Algorithms Technology Applied in the Case Study

Programming Technology Used

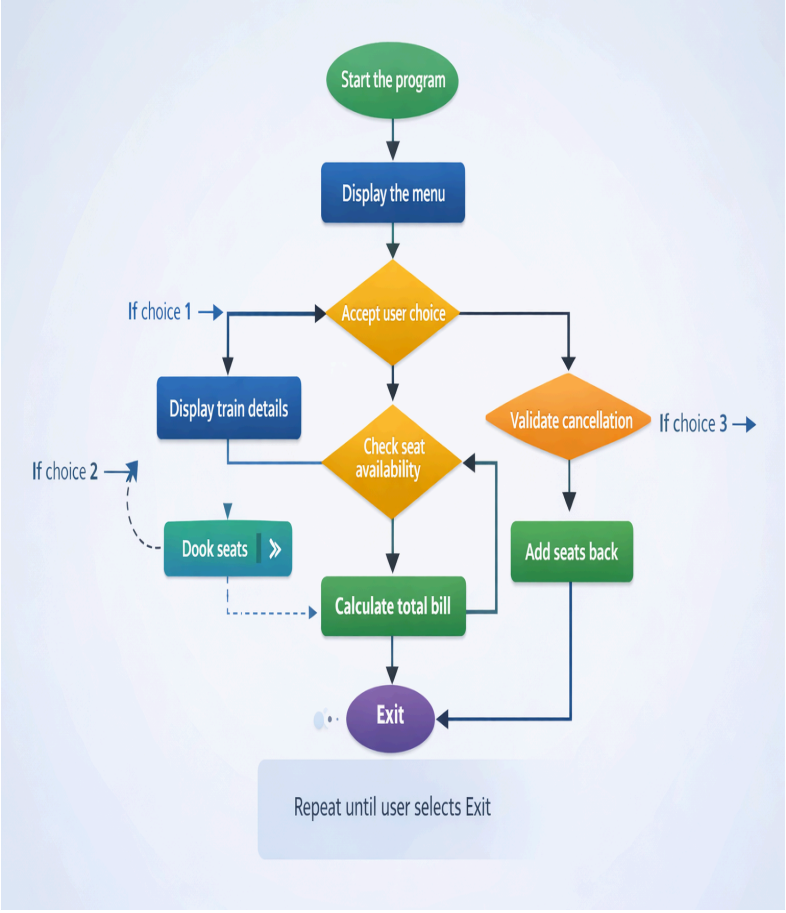
- Programming Language: C++
- Compiler: Turbo C++, Dev C++, VS Code, or any standard C++ compiler

Concepts Applied

- Arrays for structured data storage
- Loops (for loop, do-while loop)
- Conditional statements (if-else)
- Switch-case control structure
- Arithmetic operations for billing calculation

Algorithm

1. Start the program
2. Display the menu
3. Accept user choice
4. If choice = 1 → Display train details
5. If choice = 2 →
    - Check seat availability
    - Deduct seats
    - Calculate total bill
6. If choice = 3 →
    - Validate cancellation
    - Add seats back
7. Repeat until user selects Exit



5. Problem Statement / Case Study Implementation Details and Snapshots

Booking Module

The booking module checks whether the requested seats are available. If valid, seats are deducted and total billing amount is calculated using:

Total Bill = Seats × Price per Seat

```
case 2: // Book seat
    cout << "Enter Train Number: ";
    cin >> number;

    for(int i = 0; i < 2; i++) {
        if(trainNo[i] == number) {

            cout << "Enter Seats to Book: ";
            cin >> seats;

            if(seats <= availableSeats[i] && seats > 0) {

                availableSeats[i] -= seats;
                int totalBill = seats * pricePerSeat[i];
```

Screenshot of booking validation and billing code

= Seat Validation and Billing Calculation

Cancellation Module

The cancellation module ensures that cancellation does not exceed total seat capacity. If valid, seats are added back.

```
case 3: // Cancel seat
    cout << "Enter Train Number: ";
    cin >> number;

    for(int i = 0; i < 2; i++) {
        if(trainNo[i] == number) {

            cout << "Enter Seats to Cancel: ";
            cin >> seats;

            if(seats > 0 && availableSeats[i] + seats <= totalSeats[i]) {

                availableSeats[i] += seats;
                cout << "Cancellation Successful!\n";
            }
```

Screenshot of cancellation validation code

= Seat Cancellation Validation Logic

Output Snapshots

View Trains Output

```
===== Railway Reservation System =====

1. View Trains
2. Book Seat
3. Cancel Seat
4. Exit
Enter Choice: 1

Train No: 12951
Train Name: Mumbai Rajdhani
Available Seats: 25
Price per Seat: Rs.1500
---------------------------

Train No: 12002
Train Name: Shatabdi Express
Available Seats: 30
Price per Seat: Rs.1200
---------------------------
```

Screenshot of View Trains output

= Displaying Available Train Details

Booking Confirmation with Bill

```
Train No: 12951
Train Name: Mumbai Rajdhani
Available Seats: 25
Price per Seat: Rs.1500
---------------------------

Train No: 12002
Train Name: Shatabdi Express
Available Seats: 30
Price per Seat: Rs.1200
---------------------------

1. View Trains
2. Book Seat
3. Cancel Seat
4. Exit
Enter Choice: 2
Enter Train Number: 12002
Enter Seats to Book: 2

----- BILL -----
Train: Shatabdi Express
Seats Booked: 2
Price per Seat: Rs.1200
Total Amount: Rs.2400
Booking Successful!
```

Screenshot of successful booking and bill output

= Successful Booking and Bill Generation

Cancellation Confirmation

```
1. View Trains
2. Book Seat
3. Cancel Seat
4. Exit
Enter Choice: 3
Enter Train Number: 12002
Enter Seats to Cancel: 1
Cancellation Successful!
```

Screenshot of successful cancellation output

= **Successful Seat Cancellation**

6. Problem Statement / Case Study Results and Conclusion

Results

- Seat booking works correctly with proper validation
- Cancellation updates seat availability accurately
- Billing calculation functions correctly
- Menu-driven interface allows continuous user interaction

Conclusion

This case study successfully demonstrates the implementation of a Railway Reservation System using C++. The project integrates structured programming concepts to simulate real-world reservation logic.

The system maintains data consistency, validates user input, and performs booking, cancellation, and billing operations efficiently. It provides a strong foundation for building more advanced reservation systems in the future.

7. References

- C++ Programming Documentation (cplusplus.com)
- Standard C++ Textbook (Academic Reference)
- Classroom Notes and Case Study Guidelines
- Official documentation of C++ compilers